

RISC-V Mentorship Coding Challenge: RISC-V Instruction Set Explorer

Write a JavaScript or Python program that completes the following tasks. All tiers must be submitted in a single project.

Tier 1 — Instruction Set Parsing (required)

1. Read the provided `instr_dict.json` file from the RISC-V Extensions Landscape repository (<https://github.com/rpsene/riscv-extensions-landscape>)
2. Parse all instruction entries and group them by their extension tag(s)
3. Print a summary table showing each extension tag, its instruction count, and one example mnemonic
4. Identify and list any instructions that belong to more than one extension

Example output line:

`rv_zba` | 4 instructions | e.g. `SH1ADD`

Tier 2 — Cross-Reference with the ISA Manual (required)

5. Clone or fetch the official RISC-V ISA manual repository (<https://github.com/riscv/riscv-isa-manual>)
6. Scan the AsciiDoc source files (under `src/`) for references to extension names (e.g., `Zba`, `Zicsr`, `M`, `F`, etc.)
7. Cross-reference the extensions found in `instr_dict.json` against those mentioned in the manual sources
8. Report:
 - Extensions present in `instr_dict.json` but **not** mentioned in the ISA manual
 - Extensions mentioned in the ISA manual but **not** present in `instr_dict.json`
 - A count summary (e.g., "42 matched, 3 in JSON only, 5 in manual only")

Note: extension naming may differ slightly between the two sources (e.g., `rv_zba` vs `Zba`). Part of the challenge is handling this normalization.

Tier 3 — Bonus (optional, but strongly encouraged)

9. Include unit tests validating your parsing, grouping, and cross-reference logic
10. Host your solution in a public Git repository with a README that includes:
 - How to install dependencies and run the program
 - Sample output
 - Any assumptions or design decisions you made

11. Generate a text-based or visual graph showing which extensions share instructions (extensions that share at least one instruction should be connected)

Evaluation Criteria

Criteria	Weight
Correctness of output	30%
Code quality and organization	25%
Handling of edge cases and data mismatches	20%
Documentation (README, comments, design decisions)	15%
Bonus items (tests, graph, repo quality)	10%

Submission

Submit a link to your public Git repository. If you prefer not to use a public repo, submit a .zip file containing your code, README, and sample output.